articles, beginning with injection flaws.

Injection flaws are so named because when they are used, malicious attacker-supplied data flows through the application, crosses system boundaries, and gets injected into other system components. These are fairly dangerous attacks, and mostly work because a string that is harmless for PHP (or another website scripting language) can turn into a dangerous weapon when it reaches the database. Such flaws and attacks are particularly important, since they can affect any dynamic Web application that has not been tested and carefully secured against such holes. We will now take a closer look at the most often encountered injection flaw, SQL injection.

SQL injection

SQL injection is an exploit in which the attacker injects SQL code into a request to the server (generally through an HTML form input box), to gain access to the back-end database, or to make changes in it. If not sanitised properly, SQL injection attacks on your Web application could allow attackers to even ruin your website, besides extracting confidential data. Website features such as login pages, feedback forms, search pages, shopping carts, etc, that use databases, are more prone to SQL injection attacks. The attacker injects specially crafted SQL commands or statements into the form, trying to achieve various results.

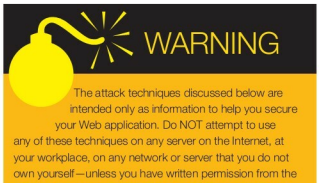
Almost all scripting technologies—ASP, ASP.NET, PHP, JSP and CGI, are vulnerable to this attack if they use MS SQL Server, Oracle, MySQL, Postgres or DB2 as their database. Basic knowledge of SQL commands, and some creative guess work, is all it takes to penetrate an unsecured application. Network firewalls and IDSs (Intrusion Detection Systems) might not help, since they provide filters on HTTP, SSL and other Web traffic ports—but the communication with the database is still unsecured. Most programmers are still not aware of the problem, and the scenario seems to be getting worse: the Web security consortium informs us that SQL injection comprised over 7 per cent of all Web vulnerabilities present in every 15,000 applications that it scanned!

Note: SQL injection is a 'global' type of attack; it is not restricted to open source platforms, databases or applications, as you can see from the inclusion of proprietary software products in the list above.

SQL injection via a login form

Let's take a common vulnerable code snippet in an ASP page, which generates a login validation query that is meant to be run on an MS SQL server database:

```
var sql = "SELECT * FROM Users WHERE usr= ' " + user + " ' AND password=' "
+ paswd + " ' ";
```



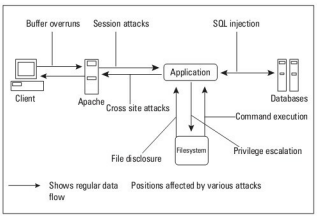


Figure 2: Typical client-server Web architecture, with attack vectors

In case a valid user enters (into the ASP login page) his username as 'arpit', and his password as 'bajpai', then the generated query becomes:

```
SELECT * FROM Users WHERE usr='arpit' AND password='bajpai'
```

That's pretty innocuous, and exactly what the person who wrote the ASP code intended. However, note what happens if an attacker submits malicious text in the username field—something like " *' or 1=1 --"* , then the query becomes…